# CSE 242 Assignment 2

## 3 Questions, 100 pts

Your name: Jiaxuan Wu Student ID: 2252665

## Instruction

- Submit your assignments onto **Canvas** by the due date. Upload a `zip` file containing:

  (1) The saved/latest `.ipynb` file.

  (2) Also save your file into a pdf version, if error appears, save an html version instead (easy to grade for written questions).

  (3) All other materials to make your `.ipynb` file runnable.

  **For assignment related questions, please reach TA or grader through EdDiscussion.**

- This is an **individual** assignment. All help from others (from the web, books other than text, genAI, or people other than the TA or instructor) must be clearly acknowledged.
- Most coding parts can be finished with only 1-2 lines of codes.
- Make sure you have installed required packages: `scikit-learn`.
- Double click to edit each markdown cell.

## Objective

- **Task 1:** Maximum likelihood estiamtion (math)
- **Task 2:** Linear Regression (with Scikit-learn)

- **Task 3:** Principle Component Analysis (with Scikit-learn)

# Question 1 (Maximum likelihood estiamtion and Posterior, 20 pts)

Assume we have a coin that has some unknown probability $h$ of coming up heads (and probability $1-h$ of coming up tails). If the coin is flipped five times getting three heads and two tails ($\mathbf{HHHTT}$) then:

(a -- 10pts) What is the maximum likelihood estimate for $h$?

(b -- 10pts) Assume that (before flipping the coin) we have a prior density $p(h)$ for the various values of $h \in [0,1]$. Give the formula for the posterior probability density $p(h \mid \mathbf{HHHTT})$ as a function of the prior $p(h)$.

For Question 1(b), the solution is acceptable if the formula contains an integral.

**Solution (a):**

Let $D = \mathbf{HHHTT}$ denote the observed sequence, and let $h \in [0,1]$ denote the probability of heads.

The flips are independent, so the likelihood of observing $\mathbf{HHHTT}$ is $L(h) = p(D \mid h) = h^3(1-h)^2$.

The log likelihood $\ell(h) = \log L(h) = 3\log h + 2\log(1-h)$.

Maximize the log likelihood by setting the derivative to zero: $\ell'(h) = \frac{3}{h} - \frac{2}{1-h} = 0 \Rightarrow\ 3(1-h) = 2h$.

The second derivative is $\ell''(h) = -3/h^2 - 2/(1-h)^2 < 0$ throughout $(0,1)$, and $L(0) = L(1) = 0$.

Therefore, $\widehat{h}_{\mathrm{MLE}} = \frac{3}{5} = 0.6$ is the unique global maximizer.

**Solution (b)**:

By Bayes' rule:

$$p(h \mid D) = \frac{P(D \mid h)\, p(h)}{P(D)} = \frac{h^3(1-h)^2\, p(h)}{\int_0^1 t^3(1-t)^2\, p(t)\, dt}$$

The denominator is the marginal likelihood $P(D) = \int_0^1 P(D \mid t)\, p(t)\, dt$, a constant that does not depend on $h$ and makes the posterior integrate to 1, i.e. $p(h \mid D) \propto h^3(1-h)^2\, p(h)$.

# Question 2 (Linear Regression, 40 pts)

In this question, you will be using **Scikit-learn** to empirically apply the Linear Regression model. We will adopt an advertisement dataset **Advertising.csv** from Kaggle.

In [1]:
```
import pandas as pd
data = pd.read_csv('Advertising.csv', index_col=0)
data.head()
```

Out[1]:

| | TV | Radio | Newspaper | Sales |
|---|---|---|---|---|
| **1** | 230.1 | 37.8 | 69.2 | 22.1 |
| **2** | 44.5 | 39.3 | 45.1 | 10.4 |
| **3** | 17.2 | 45.9 | 69.3 | 9.3 |
| **4** | 151.5 | 41.3 | 58.5 | 18.5 |
| **5** | 180.8 | 10.8 | 58.4 | 12.9 |

In [2]:
```
data.shape
```

Out[2]:
```
(200, 4)
```

## Features and responses

What are the features?

- **TV:** advertising dollars spent on TV for a single product in a given market (in thousands of dollars)
- **Radio:** advertising dollars spent on Radio
- **Newspaper:** advertising dollars spent on Newspaper

What is the response?

- **Sales:** sales of a single product in a given market (in thousands of items)

What else do we know?

- There are 200 **observations**, and each observation (each row) is a single market.
- **Your main task**: predict the sales of a single product in a given market.
- Since the response variable is continuous, this is a **regression** problem.

## Linear regression

**Pros:** fast, no tuning required, highly interpretable

**Cons:** adopting the linear regression is unlikely to always generate the best predictive accuracy, since Linear Regression presumes a linear relationship between the features and response. This assumption may not always make sense. (But if one can *transform* the features so that do have a linear relationship, sometimes progress can be made, for example by taking logs.)

**Expression:**

- $y$: the response
- $\beta_0$: the intercept
- $\beta_1$: the coefficient for $x_1$ (the first feature)
- $\beta_i$ is the coefficient for $x_i$ (the ith feature, $i \in \{1, 2, \ldots, n\}$)

For this regression task:

$$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$$

The $\beta$ values ($\beta = [\beta_0, \beta_1, \beta_2, \beta_3]$) are called the **model coefficients**. These values are "learned" during the model fitting step using the "least squares" criterion. Then, we can make use of the fitted model to make predictions!

## Prepare the dataset for training use

- Scikit-learn expects X (feature matrix) and y (response vector) to be NumPy arrays.
- Pandas is built on top of NumPy, and exposes numpy methods. Thus, X can be a pandas DataFrame, y can be a pandas Series!

In [3]:
```
# Prepare the feature X
X = data[['TV', 'Radio', 'Newspaper']]

# check the type and shape of X
print(type(X))
print(X.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
(200, 3)
```

In [4]:
```
# Prepare the response Y
y = data['Sales']
# check the type and shape of y
print(type(y))
print(y.shape)
```

```
<class 'pandas.core.series.Series'>
(200,)
```

# Question 2.1 (Split the dataset into train and test parts, 5 pts)

For features $X$ and response $y$, use **sklearn** to perform the the splitting of dataset: $80\%$ for train data, $20\%$ for the test data.

In [5]:
```
################## Your answer for Question 2.1 #################
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
################## Your code above #################
```

Double check the shape of train and test data:

In [6]:
```
# default split is 80% for training and 20% for testing
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(160, 3)
(160,)
(40, 3)
(40,)
```

## Question 2.2 (Train a Linear regression model via Scikit-learn, 10 pts)

Assume you model is named as **model**, and you train the linear regression model to fit on the training data.

In [7]:
```
################## Your answer for Question 2.2 #################
from sklearn.linear_model import LinearRegression
model = LinearRegression().fit(X_train, y_train)
################## Your code above #################
```

In [8]:
```
# Take a look at your model coefficients
import numpy as np
print(np.round(model.intercept_, 4))
print([np.round(val, 4) for val in model.coef_])
```

```
2.9949
[np.float64(0.0446), np.float64(0.1965), np.float64(-0.0028)]
```

In [9]:
```
# Match the feature names with the trained coefficients
list(zip(data.columns[:3].tolist(), model.coef_))
```

Out[9]:
```
[('TV', np.float64(0.04458402011996432)),
 ('Radio', np.float64(0.196497034155405)),
 ('Newspaper', np.float64(-0.002781463981926014))]
```

## Question 2.3 (Interpreting model coefficients, 10 pts)

The coefficients of each feature are printed above.

## Your tasks:

- What is the trained linear regression model?
- **(replace $\beta_i$ below with the trained coefficients, reserve four decimal places, 5 pts)**

$$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$$

- **(Open) question (5 pts)**: how do you interpret the coefficient of **TV** ($\beta_1$)?

## Your solution for Question 2.3:

$$\text{Sales} = 2.9949 + 0.0446 \times \text{TV} + 0.1965 \times \text{Radio} - 0.0028 \times \text{Newspaper}$$

Holding Radio and Newspaper spending fixed, an additional $1,000 spent on TV advertising is associated with an increase of approximately 0.0446 thousand items (44.6 items) in predicted sales.

## Making predictions

In [10]:
```
# make predictions on the testing set
y_pred = model.predict(X_test)
# Print the test accuracy score from your trained linear regression model
model.score(X_test,y_test)
```

Out[10]:
```
0.8601145185017869
```

# Model evaluation metrics for regression

Evaluation metrics for classification problems, such as **accuracy**, are not useful for regression problems. Instead, we need evaluation metrics designed for comparing continuous values. (Since predictions would hardly ever exactly match the measured quantities.)

**Mean Squared Error** (MSE) is the mean of the squared errors:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

**Root Mean Squared Error** (RMSE) is a popular evaluation metric for regression, which is defined as the square root of the mean of the squared errors:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

# Question 2.4 (Evaluate your predictions on test features with RMSE metric, 5 pts)

Hint: you may calculate MSE using scikit-learn, but RMSE is a little bit different from MSE.

In [11]:
```
from sklearn import metrics
################## Your answer for Question 2.4 #################
rmse = np.sqrt(metrics.mean_squared_error(y_test, y_pred))
print(f'RMSE: {rmse:.6f}')
################## Your code above #################
```

```
RMSE: 2.098123
```

# Question 2.5 (Does the feature 'Newspaper' help? 10 pts)

Does the feature **Newspaper** contribute or impair the model prediction? Now repeat previous procedure:

- For features $X$, only select **TV** and **Radio**; response $y$ is unchanged.
- Split the dataset as what you have done before.
- Fit/Train the model on the train data.
- Make predictions on the test data.

- Compute the RMSE of the predictions on the test data.
- **Show your observations!**

**Reminder:** if you use `random_state` while splitting the dataset, make sure the value of `random_state` is the same for the two splits. So that the test data only differs in the column of **Newspaper**.

In [12]:
```
################## Your answer for Question 2.5 #################
X_reduced = data[['TV', 'Radio']]
X_train_reduced, X_test_reduced, y_train_reduced, y_test_reduced = train_test_split(X_reduced, y, test_size=0.2,
random_state=0)
model_reduced = LinearRegression().fit(X_train_reduced, y_train_reduced)
y_pred_reduced = model_reduced.predict(X_test_reduced)
rmse_reduced = np.sqrt(metrics.mean_squared_error(y_test_reduced, y_pred_reduced))
print(f'RMSE: {rmse_reduced:.6f}')
################## Your code above #################
```

```
RMSE: 2.095574
```

### Your observations:

On this test split, removing Newspaper slightly lowers the test RMSE from 2.0981 to 2.0956, indicating that including Newspaper slightly impairs prediction.

# Question 3 (PCA, 40 pts)

In this question, we will explore how PCA helps with reducing the dimensionality through scikit-learn. We will adopt a simple digits dataset contained in scikit-learn.

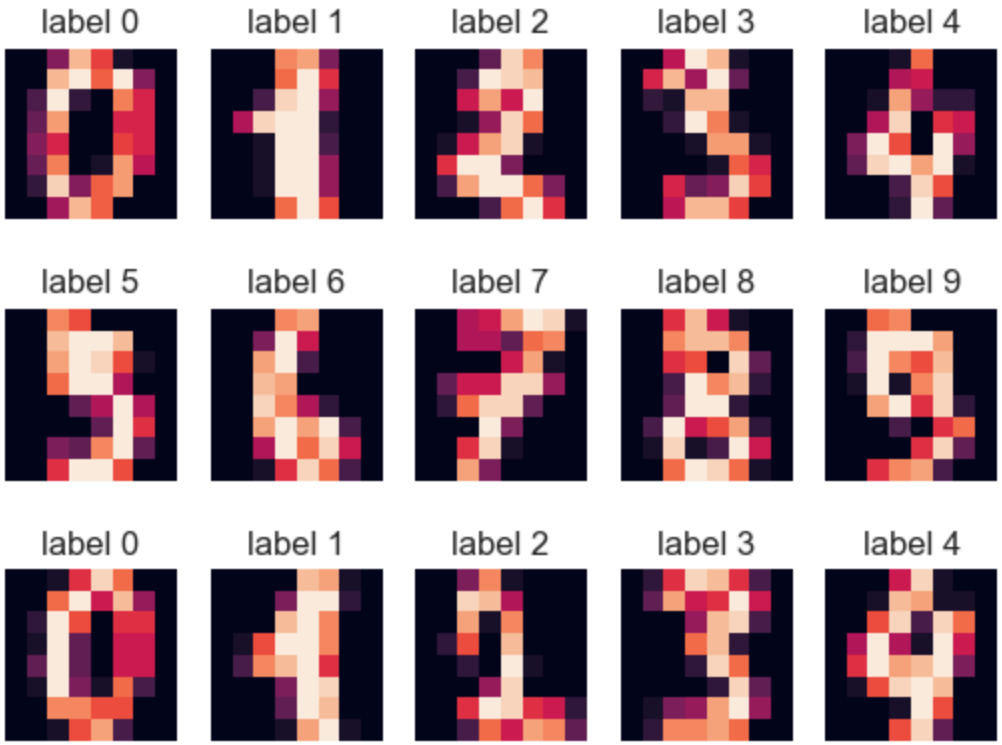
In [13]:
```
# import libraries
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns;
sns.set()
```

In [14]:
```
from sklearn.datasets import load_digits
data_digit = load_digits()
data_digit.data.shape
```

Out[14]:
```
(1797, 64)
```

As you have seen above, there are 1797 rows and 64 columns in this dataset. Each row denotes the pixel values of an image. Since each image is 8*8 (in pixel), the dimension of each feature is 64. Let's take a look at a few images contained in this dataset.

In [15]:
```
# A help function to display images with labels
def display_digits(imgs):
    h = 3
    w = 5
    fig, axes = plt.subplots(h, w)
    for i in range(h):
        for j in range(w):
            if imgs.shape[1] == 64:
                axes[i, j].imshow(imgs[i * w + j].reshape(8, 8))
            else:
                axes[i, j].imshow(imgs[i * w + j])
            axes[i, j].axis('off')
            axes[i, j].set_title(f'label {data_digit.target[i * w + j]}')
# Take a look at a few examples
display_digits(data_digit.images)
```




**PCA** finds uncorrelated orthogonal axes (principal components) in a high dimensional space (i.e., 64) to project the feature onto principal components.

# Question 3.1 (PCA and dimensionality deduction with Scikit-learn, 10 pts)

In this question, you may use the implementation of PCA from scikit-learn, as imported below. Detailed explanations of parameters settings are available in this link.

## Your task in this question:

Fit the model with feature X (**data_digit.data** for this dataset) and apply the dimensionality reduction on X, keep 2 components.

In [16]:
```
from sklearn.decomposition import PCA

################## Your answer for Question 3.1 #################
pca = PCA(n_components=2, svd_solver='full')
projected = pca.fit_transform(data_digit.data)
################## Complete the code above #################

# Check that the feature dimension is reduced from 64 to 2.
print(data_digit.data.shape)
print(projected.shape)
```

```
(1797, 64)
(1797, 2)
```

With the reduced dimension of features, we can now visualize the projected/reduced features in a 2-dimension plot as below. There are hardly any overlaps between digit 0 and any other digits. While in the central area of the figure, features are not well separable.

In [17]:
```
plt.figure(figsize=(18, 8))
plt.scatter(projected[:, 0], projected[:, 1],
            c=data_digit.target, edgecolor='none', alpha=0.7,
            cmap=plt.get_cmap('Paired', 10))
plt.xlabel('1st Component')
plt.ylabel('2nd Component')
plt.colorbar()
```

Out[17]:
```
<matplotlib.colorbar.Colorbar at 0x23071f25160>
```




## Question 3.2 (How many components do we need? 15 pts)

In practice, suppose you are given a high-dimension dataset, and you are interested in performing a dimension reduciton with PCA. How many conponents do we need? One method is to look at the distribution of the explained variance ratio w.r.t. each component.

In this question, you may use the function `explained_variance_ratio` in **pca** (with scikit-learn) to address this question. Detailed explanations of parameters settings are available in this link.

### Your task in this question:

Print the ratio of variance explained by each of the selected components;

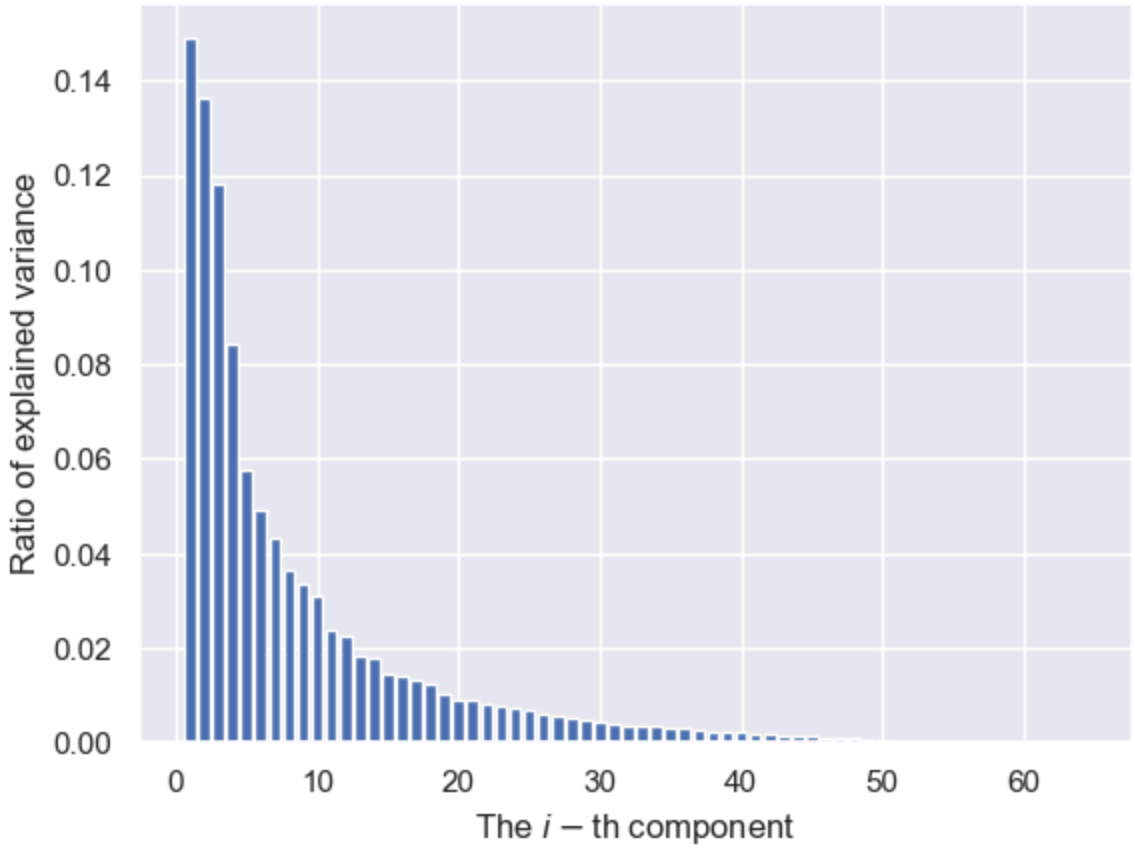
Visualize with matplotlib or seaborn (x axis: $i-th$ component; y axis: the corresponding ratio of explained variance).

In [18]:
```
################## Your answer for Question 3.2   #################
pca = PCA().fit(data_digit.data)
ratio = pca.explained_variance_ratio_
print(ratio)
plt.bar(np.arange(1, len(ratio) + 1), ratio)

plt.xlabel('The $i-$th component')
plt.ylabel('Ratio of explained variance')
################## Complete the code above #################
```

```
[1.48905936e-01 1.36187712e-01 1.17945938e-01 8.40997942e-02
 5.78241466e-02 4.91691032e-02 4.31598701e-02 3.66137258e-02
 3.35324810e-02 3.07880621e-02 2.37234084e-02 2.27269657e-02
 1.82186331e-02 1.77385494e-02 1.46710109e-02 1.40971560e-02
 1.31858920e-02 1.24813782e-02 1.01771796e-02 9.05617439e-03
 8.89538461e-03 7.97123157e-03 7.67493255e-03 7.22903569e-03
 6.95888851e-03 5.96081458e-03 5.75614688e-03 5.15157582e-03
 4.89539777e-03 4.28887968e-03 3.73606048e-03 3.53274223e-03
 3.36683986e-03 3.28029851e-03 3.08320884e-03 2.93778629e-03
 2.56588609e-03 2.27742397e-03 2.22277922e-03 2.11430393e-03
 1.89909062e-03 1.58652907e-03 1.51159934e-03 1.40578764e-03
 1.16622290e-03 1.07492521e-03 9.64053065e-04 7.74630271e-04
 5.57211553e-04 4.04330693e-04 2.09916327e-04 8.24797098e-05
 5.25149980e-05 5.05243719e-05 3.29961363e-05 1.24365445e-05
 7.04827911e-06 3.01432139e-06 1.06230800e-06 5.50074587e-07
 3.42905702e-07 0.00000000e+00 0.00000000e+00 0.00000000e+00]
```

Out[18]:
```
Text(0, 0.5, 'Ratio of explained variance')
```




## How to obtain eigen values and vectors from sklearn PCA

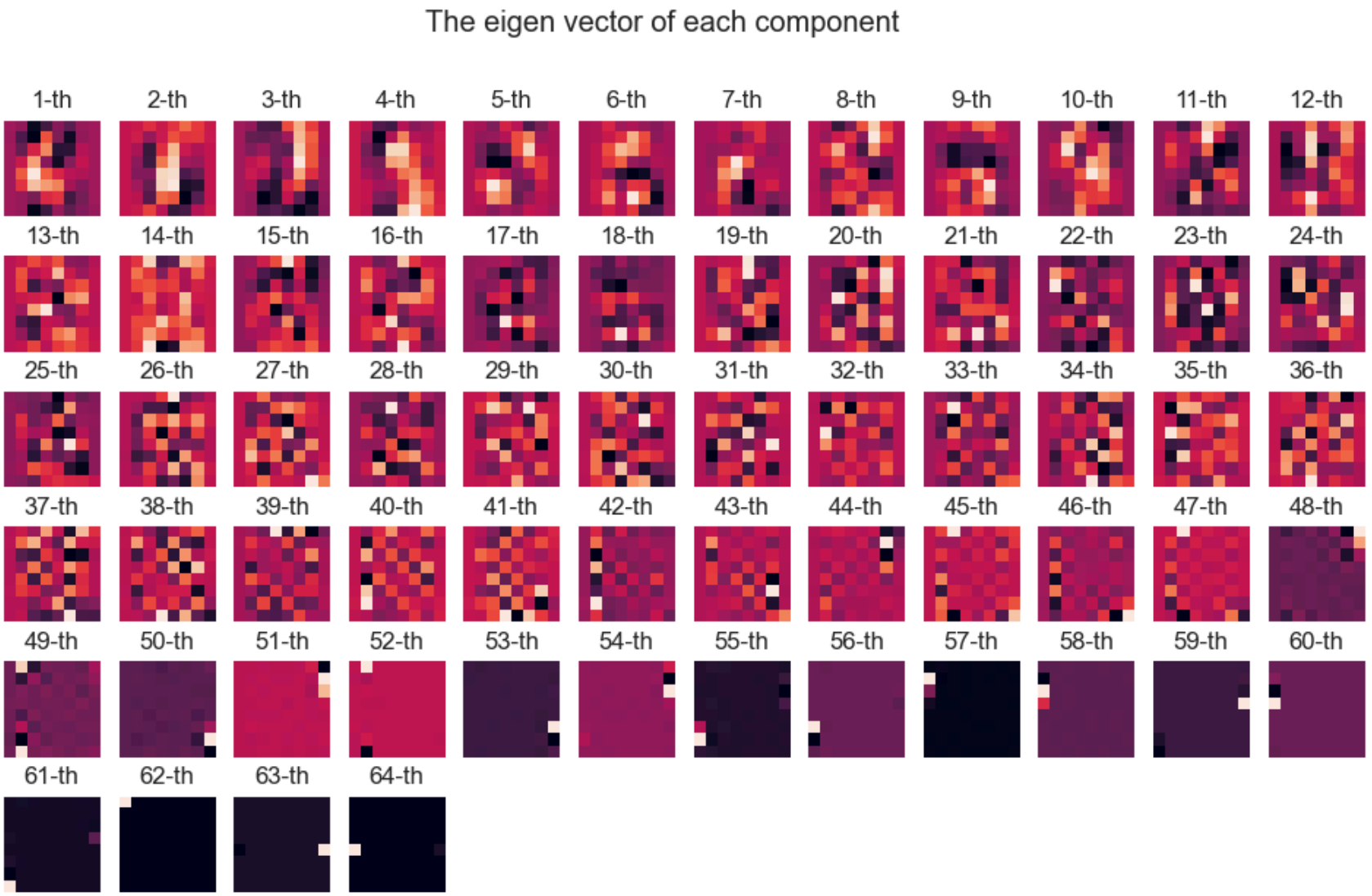
- Eigen values -- `your-PCA-model.explained_variance_`
- Eigen vectors -- `your-PCA-model.components_`

## Eigen vectors (Eigen digits)

In [19]:
```
# Default # of components is the dimension of feature space
pca = PCA()
pca.fit(data_digit.data)
h = 6
```

```
w = 12
fig, axes = plt.subplots(h, w)
fig.set_size_inches(12, 7)
for i in range(h):
    for j in range(w):
        idx = i * w + j
        if idx > 63:
            axes[i, j].axis('off')
        else:
            axes[i, j].imshow(pca.components_[idx].reshape(8, 8))
            axes[i, j].axis('off')
            axes[i, j].set_title(f'{idx + 1}-th')
fig.suptitle('The eigen vector of each component')
```

Out[19]: Text(0.5, 0.98, 'The eigen vector of each component')




Question 3.3 What are your observations from the visualized eigen vectors? (5 pts)

Your solutions for Question 3.3:

The first few eigenvectors show broad, stroke-like patterns concentrated near the center of the image. Later eigenvectors show finer, more alternating patterns that are harder to recognize as digit strokes. Many of the last eigenvectors have noticeable weights at only a few pixels, often near the image border.

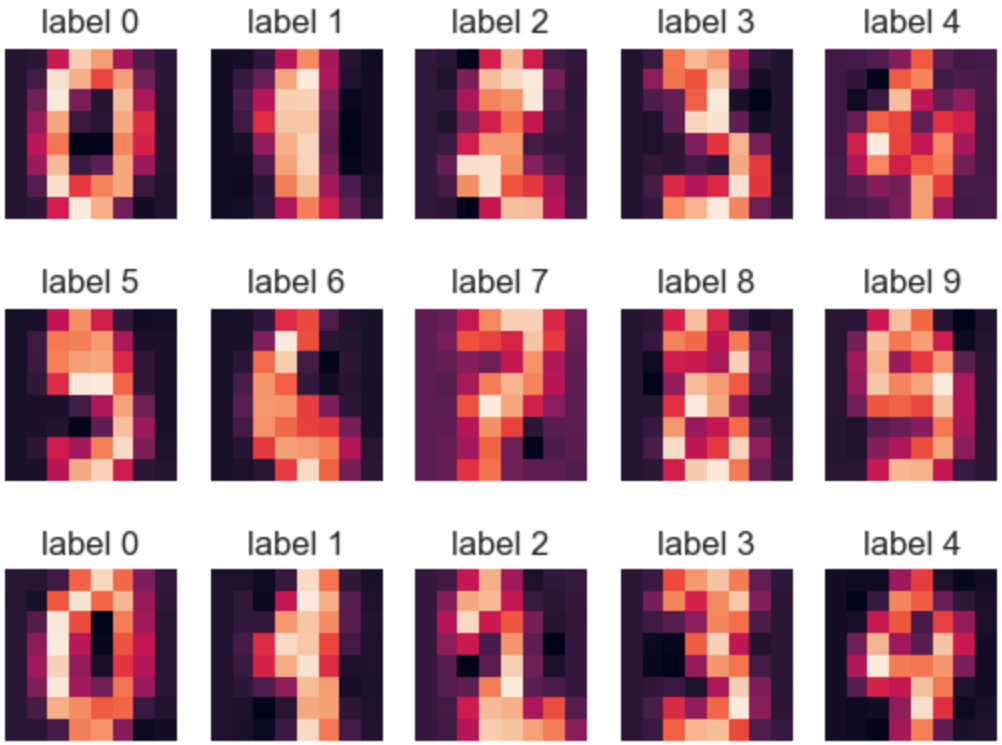
## Feature reconstruction:

Suppose you only want to preserve $80\%$ variance after applying PCA for the dimension reduction, due to storage issues, etc.

In [20]:
```
# Use pca from sklearn to fit on the feature space
pca_08 = PCA(0.8).fit(data_digit.data)

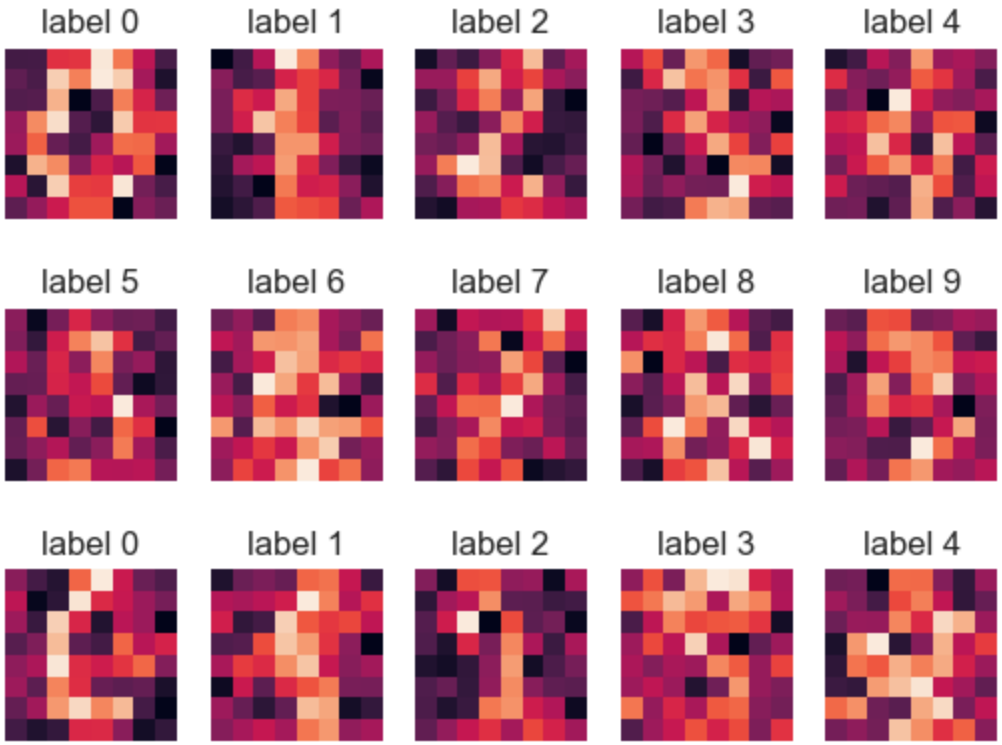
# Reconstruct
lower_dimension = pca_08.fit_transform(data_digit.data)
reconstructed = pca_08.inverse_transform(lower_dimension)

# Display the reconstructed images (with a much lower dimension)
display_digits(reconstructed)
```




Assume now you are given only a set of perturbed images (with some random noise). As shown below, the figures become much more difficult to recognize.

In [21]:
```
# Apply random noise on clean images
noisy_imgs = np.random.normal(data_digit.data, 5)
# Display the noisy images
display_digits(noisy_imgs)
```

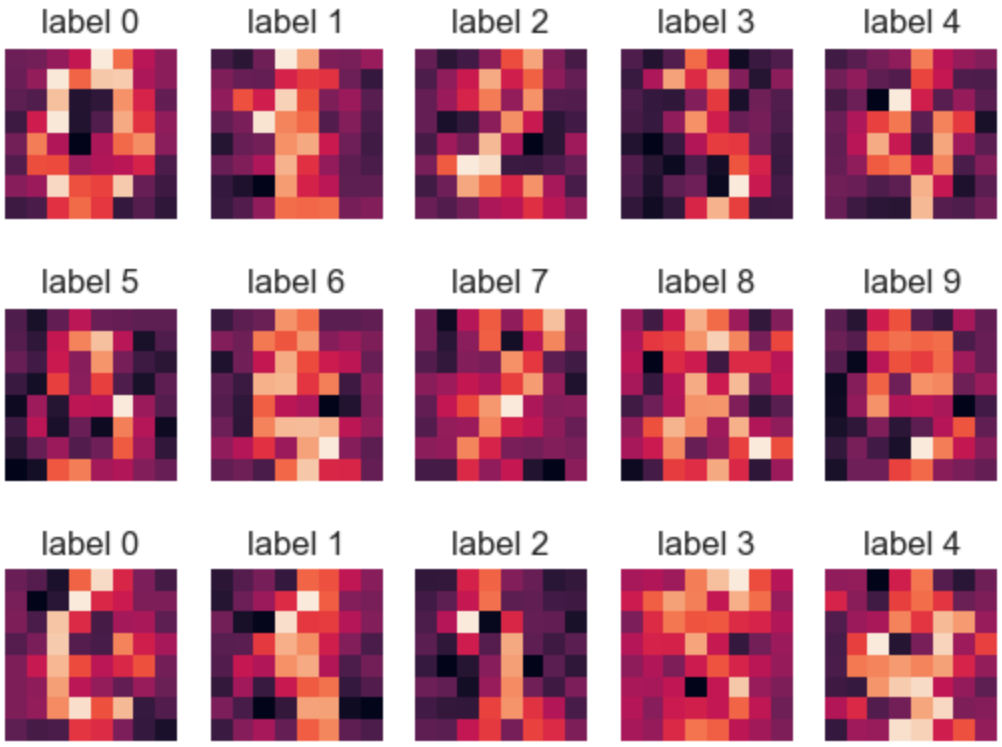



## Question 3.4 Does PCA mitigate the random noise? (10 pts)

### Your tasks:

- Perform feature reconstruction on `noisy_imgs` (preserving $80\%$ variance after applying PCA for the dimension reduction)
- Explain your understandings on why PCA reconstructs cleaner images.

In [22]:
```
################## Your answer for Question 3.4   ##################
pca_noise = PCA(0.8).fit(noisy_imgs)
reconstructed_noise = pca_noise.inverse_transform(pca_noise.fit_transform(noisy_imgs))
display_digits(reconstructed_noise)
################## Complete the code above ##################
```




In [23]:
```
print('# components kept:', pca_noise.n_components_)
print(f'MSE before denoising: {metrics.mean_squared_error(data_digit.data, noisy_imgs):.6f}')
print(f'MSE after denoising: {metrics.mean_squared_error(data_digit.data, reconstructed_noise):.6f}')
```

```
# components kept: 40
MSE before denoising: 25.021751
MSE after denoising: 17.446595
```

## Your written answers for Question 3.4:

PCA mitigates the noise and retains the components that account for at least 80% of the variance in the noisy images. The leading components capture much of the shared structure of the digits, while independent Gaussian noise is spread across all directions. Reconstructing from the retained components removes the variation in discarded directions, including some of the noise, so the images become cleaner. Noise within the retained directions remains.

# AI accreditation

ChatGPT and Claude were used to help interpret the requirements and check/polish my answers.

https://chatgpt.com/share/6ada9637-c9b0-83e8-a95c-f1c35a48c5b8

https://claude.ai/artifact/EiAubdchGLLbSwU5kNkRDA